

知识点1 Linux 文件编程

在Linux系统中一切皆文件，包括常规或者设备等，均以文件形式进行交互。

1 文件编程的基础

1.1 ls命令

查看文件命令：

ls 输出当前目录中的文件名名称；等价于: ls .

ls -a 输出当前目录中的所有文件名称，包含隐藏文件（是以“.”开头的文件）；

ls -l 输出当前目录中所有文件的属性

ls 文件名称/目录名称 -a -l

1.2 文件描述符

所谓的文件描述符，用来标识进程中所打开文件的ID标识符。

1) 是一个非负整数；

2) 分配是在文件打开的时候，由系统进程进行分配；将系统进程中位被使用的最小非负整数分配给当前打开的文件；

3) 在进程启动的过程中，系统默认分配三个文件描述符：

0 = 标准输入； 1 = 标准输出； 2 = 标准错误输出

2 IO操作的函数

2.1 文件的打开：open

1. open函数原型

```
1  #include <sys/types.h>
2  #include <sys/stat.h>
3  #include <fcntl.h>
4
5  int open(const char *pathname, int flags);
6  int open(const char *pathname, int flags, mode_t mode);
7  功能：打开已存在的文件或者创建不存在的文件同时打开；
8  参数：
9      pathname: 所有打开文件的路径名称（相对路径或者绝对路径）
10     flags: 所打开文件的方式，使用预定义的标识符常量设置，至少包含一个或者多个(多个使用运算符"|")
11           以下三个包含且仅包含其中一个，用于设置打开文件的访问权限
12           O_RDONLY      : 只读
13           O_WRONLY      : 只写
14           O_RDWR       : 可读写
```

```
15         可选：
16         O_CREAT          ： 创建文件，文件不存在则创建并打开，文件已存在直接打开源文件，
源文件数据保留，写文件的时候将源文件内容顺序替换。
17         O_CREAT | O_EXCL   ： 创建文件，文件不存在则创建文件，文件已存在则报错，
18         O_CREAT | O_TRUNC   ： 创建文件，文件不存在则创建文件，文件已存在打开源文件并将源文
件内容清除；
19         O_CREAT | O_APPEND  ： 创建文件，文件不存在则创建文件，文件已存在打开源文件，保留原
文件内容，并且将写文件指针偏移 to 文件末尾；
20
21         mode: 只有在新创建文件的时候有效，设置所创建文件的访问权限，使用八进制数据表示（0777）
22 返回值：
23         成功返回文件的文件描述符ID；失败返回-1且修改errno的值。
24
```

2. open函数使用实例

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <sys/types.h>
4  #include <sys/stat.h>
5  #include <fcntl.h>
6  #include <unistd.h>
7
8  int main(int argc, char *argv[])
9  {
10     int fd;
11     /* 以读写方式打开文件，文件不存在则创建文件并设置文件的访问权限为0777，文件已存在则直接打开
文件 */
12     fd = open(argv[1], O_RDWR | O_CREAT, 0777);
13     if (fd == -1) {
14         perror("open");
15         return -1;
16     }
17 }
```

2.2 读文件：read

1. read函数原型

```
1 #include <unistd.h>
2 ssize_t read(int fd, void *buf, size_t count);
3 功能：读文件
4 参数：
5     fd：所有读文件的文件描述符ID；
6     buf：缓存空间的起始地址，缓存空间用于存储所读取文件的数据内容；
7     count：设置 参数2 buf指针指向空间的字节数，用于表示当前期望读取数据的字节数；
8 返回值：
9     成功读取数据返回读取数据的字节数；
10    返回值为0，表示读取到文件结束。
11    失败返回-1，且修改errno值
12
```

2. read读文件实例

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <un:seristd.h>
7
8 int main(int argc, char *argv[])
9 {
10     int fd;
11     char buf[32];
12     int ret;
13     /* 读写方式打开文件 */
14     fd = open(argv[1], O_RDWR);
15     if (fd == -1) {
16         perror("open");
17         return -1;
18     }
19     /* 循环读取文件数据并标准输出 */
20     while(1) {
21         memset(buf, 0, sizeof(buf));
22         ret = read(fd, buf, sizeof(buf));
23         if (ret == -1) {
24             perror("read");

```

```

25         return -1;
26     } else if (ret == 0) {
27         printf("read end of file\n");
28         break;
29     }
30     printf("%s\n-----\n", buf);
31 }
32 }

```

2.3 写文件：write

1. write函数原型

```

1  #include <unistd.h>
2  ssize_t write(int fd, const void *buf, size_t count);
3  功能：写文件
4  参数：
5      fd：所有写文件的文件描述符ID；
6      buf：缓存空间的起始地址，缓存空间存储所有写入文件中的数据；
7      count：设置 参数2 buf指针指向缓存空间的数据字节数，用于表示当前期望写入文件的数据字节数。
8  返回值：
9      成功返回写入文件的字节数；失败返回-1且设置errno值。

```

2. 写文件实例

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <sys/types.h>
4  #include <sys/stat.h>
5  #include <fcntl.h>
6  #include <unistd.h>
7
8  int main(int argc, char *argv[])
9  {
10     int fd;
11     int ret;
12
13     /* 以读写方式打开文件，文件不存在则创建文件并设置文件的访问权限为0777，文件已存在则直接打开
文件 */

```

```

14     fd = open(argv[1], O_RDWR | O_CREAT, 0777);
15     if (fd == -1) {
16         perror("open");
17         return -1;
18     }
19
20     ret = write(fd, "abc", 3);
21     if (ret == -1) {
22         perror("write");
23         return -1;
24     }
25 }

```

2.4 文件操作练习

实现文件的拷贝功能

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <sys/types.h>
4  #include <sys/stat.h>
5  #include <fcntl.h>
6  #include <un:seriStd.h>
7
8  int main(int argc, char *argv[])
9  {
10     int src_fd;
11     int dest_fd;
12     int ret;
13     char buf[128];
14
15     /* 打开源文件 */
16     src_fd = open(argv[1], O_RDWR);
17     if (src_fd == -1) {
18         perror("open->src_file");
19         return -1;
20     }
21
22     /* 打开目标文件：文件不存在则创建，文件存在则清除源文件中数据 */
23     dest_fd = open(argv[2], O_RDWR | O_CREAT | O_TRUNC, 0777);

```

```

24     if (dest_fd == -1) {
25         perror("open->dest_file");
26         close(src_fd);
27         return -1;
28     }
29
30     while(1) {
31         memset(buf, 0, sizeof(buf));
32         /* 读取源文件数据 */
33         ret = read(src_fd, buf, sizeof(buf));
34         if (ret == -1) {
35             perror("read->src_file");
36             close(dest_fd);
37             close(src_fd);
38             return -1;
39         } else if (ret == 0) {
40             break;
41         }
42         /* 写数据到目标文件中 */
43         ret = write(dest_fd, buf, ret);
44         if (ret == -1) {
45             perror("write->dest_file");
46             close(dest_fd);
47             close(src_fd);
48             return -1;
49         }
50     }
51     close(dest_fd);
52     close(src_fd);
53     return 0;
54 }

```

知识点2 多线程编程

1 多任务的概述

1. 任务的定义

任务是逻辑的概念，指的是由一个软件完成，或者一系列共同达到的某一操作。

2. 多任务的定义

指的是通过调度策略实现多个任务的并发执行。

3. 并行和并发

所谓并行，指的是多个任务同时不同的CPU中运行的过程，称为多任务并行；

所谓并发，指的是多个任务同时在同一个CPU中运行的过程，注意CPU在同一时间只能运行一个任务，所以此时的多个任务通过时间片轮转机制实现多任务的并发。

4. Linux系统中的多任务实现

Linux系统中的多任务实现主要采用多进程和**多线程**两种方式实现。

2. 多线程的管理及API接口

2.1 多线程的创建

1. pthread_create函数原型

```
1 #include <pthread.h>    /* 头文件 */
2 int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
3                     void *(*start_routine) (void *), void *arg);
4 功能：创建新的子线程，设置子线程创建成功的线程回调函数；
5 参数：
6     thread：子线程创建成功的时候返回子线程的线程ID；
7     attr：设置所创建子线程的线程属性，一般设置为NULL表示使用默认线程属性；
8     start_routine：函数指针，设置子线程回调函数的地址，设置子线程创建成功的回调函数；
9     arg：是给回调函数所传递的参数。
10 返回值：
11     成功返回0， 失败返回errno
12 Compile and link with -pthread.    /* 程序编译的时候需要链接库 */
```

2. 子线程创建实例

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <pthread.h>
4
5 /* 自定义线程执行函数 */
6 void *thread1_func(void *arg)
7 {
8     while(1) {
9         printf("thread 1\n");
10        sleep(1);
11    }
12 }
13
```

```

14 void *thread2_func(void *arg)
15 {
16     while(1) {
17         printf("thread 2\n");
18         sleep(2);
19     }
20 }
21
22 int main(int argc, char *argv[])
23 {
24     pthread_t thread1_id;
25     pthread_t thread2_id;
26
27     /* 创建子线程1，设置线程回调函数 */
28     if (0 != pthread_create(&thread1_id, NULL, thread1_func, NULL)) {
29         fprintf(stderr, "create thread 1 fail\n");
30         return -1;
31     }
32
33     /* 创建子线程2，设置线程回调函数 */
34     if (0 != pthread_create(&thread2_id, NULL, thread2_func, NULL)) {
35         fprintf(stderr, "create thread 2 fail\n");
36         return -1;
37     }
38
39     while(1) {
40         printf("main thread\n");
41         sleep(10);
42     }
43 }

```

3. 线程创建的参数的传递

使用pthread_create的第4个参数给线程回调函数传递参数，需要根据所传递参数的数据类型在线程回调函数中进行数据的解析。

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <pthread.h>
4
5 struct Stu {

```



```
6         int num;
7         char name[32];
8         int age;
9     };
10
11     /* 自定义线程执行函数 */
12     void *thread1_func(void *arg)
13     {
14         while(1) {
15             printf("thread 1 %d\n", *(int *)arg);
16             sleep(1);
17         }
18     }
19
20     void *thread2_func(void *arg)
21     {
22         while(1) {
23             printf("thread 2 :%d-%s-%d\n", ((struct Stu *)arg)->num, ((struct Stu
24 *)arg)->name, ((struct Stu *)arg)->age);
25             sleep(2);
26         }
27     }
28
29     int main(int argc, char *argv[])
30     {
31         pthread_t thread1_id;
32         pthread_t thread2_id;
33         int a = 3;
34         struct Stu stu = {1, "kunkun", 18};
35
36         /* 创建子线程1, 设置线程回调函数 */
37         if (0 != pthread_create(&thread1_id, NULL, thread1_func, (void *)&a)) {
38             fprintf(stderr, "create thread 1 fail\n");
39             return -1;
40         }
41
42         /* 创建子线程2, 设置线程回调函数 */
43         if (0 != pthread_create(&thread2_id, NULL, thread2_func, (void *)&stu)) {
44             fprintf(stderr, "create thread 2 fail\n");
45             return -1;
46         }
```

```
47
48     while(1) {
49         printf("main thread\n");
50         sleep(10);
51     }
52 }
```

4. 子线程创建成功，子线程和主线程执行顺序是随机执行

5. 子线程的资源

1) 子线程**共享**主线程中的资源数据

所定义的全局变量资源数据；

子线程创建之前所定义的局部变量数据可以通过子线程创建的时候传递地址方式实现资源共享；

2) 子线程私有资源数据

指的是在子线程中所创建的局部变量。

2.2 多线程的其它管理

2.3.1 线程的退出

1. pthread_exit函数原型

```
1 #include <pthread.h>
2 void pthread_exit(void *retval);
3 功能：终止调用线程，也就是线程中调用的时候结束调用线程（自杀）
4 参数：retval：线程结束返回数据的起始地址
5 Compile and link with -pthread.
6 注意：只会结束调用的当前线程，对其他线程无影响。
```

2.3.2 线程阻塞等待

1. pthread_join函数原型

```
1 #include <pthread.h>
2 int pthread_join(pthread_t thread, void **retval);
3 功能：在主线程中调用，阻塞等待线程ID为thread的子线程退出；并接收子线程退出的返回资源数据，还对子线程资源的回收处理
4 Compile and link with -pthread.
```

2.3.3 线程的取消

1. pthread_cancel函数原型

```
1 #include <pthread.h>
2 int pthread_cancel(pthread_t thread);
3 功能：在主线程中调用，取消线程ID为thread的子线程退出（他杀）。
4 Compile and link with -pthread.
```

2.3.4 线程的分离

1. pthread_detach函数原型

```
1 #include <pthread.h>
2 int pthread_detach(pthread_t thread);
3 功能：主线程中调用，将线程ID为thread的子线程从主线程中进行分离，分离后子线程退出，子线程资源由自己回收处理。
4 Compile and link with -pthread.
```

2.3.5 线程综合管理实例

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <pthread.h>
4
5 struct Stu {
6     int num;
7     char name[32];
8     int age;
9 };
10
11 /* 自定义线程执行函数 */
12 void *thread1_func(void *arg)
13 {
14     while(1) {
15         printf("thread 1 %d\n", *(int *)arg);
16         sleep(1);
17     }
```

```

18 }
19
20 void *thread2_func(void *arg)
21 {
22     while(1) {
23         printf("thread 2 :%d-%s-%d\n", ((struct Stu *)arg)->num, ((struct Stu *)arg)-
>name, ((struct Stu *)arg)->age);
24         sleep(5);
25         pthread_exit(NULL);
26     }
27 }
28
29 int main(int argc, char *argv[])
30 {
31     pthread_t thread1_id;
32     pthread_t thread2_id;
33     int a = 3;
34     struct Stu stu = {1, "kunkun", 18};
35
36     /* 创建子线程1，设置线程回调函数 */
37     if (0 != pthread_create(&thread1_id, NULL, thread1_func, (void *)&a)) {
38         fprintf(stderr, "create thread 1 fail\n");
39         return -1;
40     }
41
42     /* 对子线程1进行线程分离 */
43     if (0 != pthread_detach(thread1_id)) {
44         fprintf(stderr, "detach thread 1 fail\n");
45         return -1;
46     }
47
48     /* 创建子线程2，设置线程回调函数 */
49     if (0 != pthread_create(&thread2_id, NULL, thread2_func, (void *)&stu)) {
50         fprintf(stderr, "create thread 2 fail\n");
51         return -1;
52     }
53     /* 主线程中阻塞等待线程ID为thread2_id的子线程退出 */
54     pthread_join(thread2_id, NULL);
55     /* 主线程中取消线程ID为thread1_id的子线程 */
56     pthread_cancel(thread1_id);
57     while(1) {
58         printf("main thread\n");

```

```
59         sleep(10);
60     }
61 }
```

3 同步和互斥

3.1 同步和互斥相关的概念

1. 临界资源

所谓的临界资源，指的是多任务中，能够同时被多个任务所访问的资源，称为临界资源。

多任务中，对于临界资源的访问存在竞态（同时被多个任务访问），导致程序的异常。

2. 互斥：

所谓的互斥，指的是临界资源的互斥，也就是说，对于临界资源的访问能同时被多个任务访问，每一个任务的访问要作为原子访问（只要被一个任务访问，就需要保证不被其它任务打断）。

在Linux系统提供线程锁实现。线程锁也称为互斥锁，值包含0和1，初始值为1表示资源可以被访问

在资源访问之前上锁：

判断互斥锁的值，值为1上锁(值-1)成功；否则值为0则阻塞等待，直到值为1上锁成功解除阻塞继续顺序执行。

在资源访问结束解锁：

直接将互斥锁的值+1。

3. 同步

所谓的同步，指的是多个任务按照特定的先后顺序执行，称为多任务的同步。

在Linux系统中提供信号量实现多任务的同步。

信号量是非负整数，信号量的值==0，表示无资源可以访问；信号量的值>0表示有资源可以访问，并且可以资源数==信号量值。

假设有读写任务：他们执行顺序为写任务执行之后，才能够执行读任务具体实现

写任务中（V操作）：

写任务的执行，

判断是否有任务被当前信号量所阻塞

如果有任务被阻塞，唤醒被阻塞的任务；如果没有任务被阻塞，将信号量的值+1；

读任务中（P操作）：

判断信号量的值是否大于0

如果大于0，表示有资源可以访问，将信号量的值-1，执行读任务；否则如果信号量值不大于0（==0）任务进入阻塞模式。

3.2 互斥锁解决资源的互斥的问题

3.2.1 互斥锁相关的API接口

1. 互斥锁的初始化函数

```
1 #include <pthread.h>
```

```
2 int pthread_mutex_init(pthread_mutex_t *restrict mutex,
3                          const pthread_mutexattr_t *restrict attr);
4 功能：初始化互斥锁
5 参数：
6     mutex：所要初始化互斥锁地址
7     attr：初始化互斥锁的属性，一般设置为NULL，表示使用默认属性。
8 注意：不要设置互斥锁的初始值，是因为必须要将互斥锁的值设置为1.
```

2. 互斥锁的上锁和解锁函数

```
1 #include <pthread.h>
2 int pthread_mutex_lock(pthread_mutex_t *mutex);
3 功能：阻塞模式上锁（在互斥锁的值为1的上，直接上锁成功，互斥锁的值为0阻塞等待）
4 int pthread_mutex_trylock(pthread_mutex_t *mutex);
5 功能：非阻塞模式上锁（在互斥锁的值为1的上，直接上锁成功，互斥锁的值为0也会立即返回，返回上锁失败错误信息）
6 int pthread_mutex_unlock(pthread_mutex_t *mutex);
7 功能：解锁
```

3.2.2 互斥锁实现资源互斥实例

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <pthread.h>
4
5 /* 定义全局变量的数组：是子线程1和子线程2的共享资源数据 */
6 int arr[10] = {1,2,3,4,5,6,7,8,9,10};
7 /* 定义线程锁 */
8 pthread_mutex_t mutex;
9
10 /* 自定义线程执行函数 */
11 void *thread1_func(void *arg)
12 {
13     int i;
14     while(1) {
15         pthread_mutex_lock(&mutex);    /* 上锁 */
16         /* 遍历整个数组集合空间的所有数据元素 */
17         for (i = 0; i < sizeof(arr)/sizeof(arr[0]); i++) {
```

```

18         printf("%d ", arr[i]);
19     }
20     pthread_mutex_unlock(&mutex);    /* 解锁 */
21     printf("\n");
22     //sleep(1);
23 }
24 }
25
26 void *thread2_func(void *arg)
27 {
28     int i;
29     int temp;
30     while(1) {
31         pthread_mutex_lock(&mutex);    /* 上锁 */
32         /* 数组集合元素的倒序存储 */
33         for (i = 0; i < (sizeof(arr)/sizeof(arr[0]))/2; i++) {
34             temp = arr[i];
35             arr[i] = arr[sizeof(arr)/sizeof(arr[0])-1-i];
36             arr[sizeof(arr)/sizeof(arr[0])-1-i] = temp;
37         }
38         pthread_mutex_unlock(&mutex);    /* 解锁 */
39     }
40 }
41
42 int main(int argc, char *argv[])
43 {
44     pthread_t thread1_id;
45     pthread_t thread2_id;
46
47     /* 初始化线程锁 */
48     pthread_mutex_init(&mutex, NULL);
49
50     /* 创建子线程1, 设置线程回调函数 */
51     if (0 != pthread_create(&thread1_id, NULL, thread1_func, NULL)) {
52         fprintf(stderr, "create thread 1 fail\n");
53         return -1;
54     }
55
56     /* 对子线程1进行线程分离 */
57     if (0 != pthread_detach(thread1_id)) {
58         fprintf(stderr, "detach thread 1 fail\n");
59         return -1;

```

```

60     }
61
62     /* 创建子线程2，设置线程回调函数 */
63     if (0 != pthread_create(&thread2_id, NULL, thread2_func, NULL)) {
64         fprintf(stderr, "create thread 2 fail\n");
65         return -1;
66     }
67     /* 对子线程2进行线程分离 */
68     if (0 != pthread_detach(thread2_id)) {
69         fprintf(stderr, "detach thread 2 fail\n");
70         return -1;
71     }
72
73     while(1) {
74         printf("main thread\n");
75         sleep(10);
76     }
77 }

```

3.3 信号量实现任务同步

3.3.1 信号量相关API接口

1. 信号量初始化函数

```

1  #include <semaphore.h>
2  int sem_init(sem_t *sem, int pshared, unsigned int value);
3  功能：初始化信号量
4  参数：
5      sem: 需要初始化信号量的指针；
6      pshared: 信号量的作用范围，设置为0表示多线程中使用；
7      value: 信号量的初始值。
8  Link with -pthread.

```

2. P操作函数

```

1  #include <semaphore.h>
2  int sem_wait(sem_t *sem);
3  功能：阻塞模式P操作

```



```
4 int sem_trywait(sem_t *sem);
5 功能：非阻塞模式的P操作
6 int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
7 功能：定时模式的P操作
```

3. V操作函数

```
1 #include <semaphore.h>
2 int sem_post(sem_t *sem);
3 Link with -pthread.
```

知识点3 网络通信编程

1 网络通信模型

1.1 OSI网络模型

OSI (Open System Interconnection) 七层模型是一个国际标准化组织 (ISO) 制定的**计算机网络通信协议的框架**，它将网络通信的过程划分为七个层次，每一层都负责不同的功能，从物理连接到应用程序的处理

应用层 (Application Layer)

提供用户接口和应用程序之间的通信服务，用户可以访问网络应用程序，如电子邮件、文件传输和远程登录。

表示层 (Presentation Layer)

负责数据的格式化、加密和压缩，确保数据在不同系统之间的交换是有效的和安全的。

会话层 (Session Layer)

管理应用程序之间的通信会话，负责建立、维护和终止会话。

传输层 (Transport Layer)

为应用程序提供**端到端**的数据传输服务，负责数据的分段、传输控制、错误恢复和流量控制。

网络层 (Network Layer)

负责数据包的路由和转发，以及网络中的寻址和拥塞控制。

数据链路层 (Data Link Layer)

提供点对点的数据传输服务，负责将原始比特流转换为数据帧，并检测和纠正传输中出现的错误。

物理层 (Physical Layer)

在物理媒介上传输原始比特流，定义了连接主机的硬件设备和传输媒介的规范。

注意：OSI模型是一个理想化模型，实现难度很大；

1.2 TCP/IP模型

TCP/IP四层模型是**互联网通信的基础框架**，它定义了网络通信的四个层次：应用层、传输层、网络层和网络接口层。这个模型简化了OSI七层模型，使得网络通信更高效、成本更低。

应用层

应用层位于TCP/IP模型的最顶层，直接为应用进程提供服务。它包括各种高层协议，如HTTP、FTP、SMTP等，这些协议负责处理用户与网络应用程序之间的通信。例如，HTTP协议用于Web浏览器与服务器之间的数据传输，而SMTP协议用于电子邮件的发送。

传输层

传输层是TCP/IP模型的第二层，提供端到端的数据传输服务。它主要使用TCP（传输控制协议）和UDP（用户数据报协议）两种协议。TCP提供可靠的数据传输服务，保证数据的顺序和完整性，适用于如文件传输、电子邮件等场景。UDP则提供无连接的快速数据传输服务，适用于实时应用，如在线视频和语音通话。

网络层

网络层位于第三层，负责数据包的路由和转发。它使用IP协议来定义数据包的传输路径，并处理不同网络之间的通信。网络层的协议还包括ICMP（用于传递控制消息和错误信息）和ARP（用于将IP地址映射为MAC地址）。

网络接口层

网络接口层是模型的第四层，它结合了OSI模型的数据链路层和物理层的功能。这一层负责管理网络硬件设备和物理媒介之间的通信，包括以太网、Wi-Fi等技术。网络接口层确保数据能够在物理网络上准确无误地传输。

2. 网络通信基础

2.1 传输层协议

传输层是提供端对端的数据传输服务，主要包含：TCP通信协议和UDP通信协议

端对端的通信：实质实现程序进程与程序进程之间的数据通信。

1. TCP通信协议

TCP通信协议，也称为流式套接字传输控制协议。

面向连接可靠传输控制协议

- 1) 通信过程：建立通信连接(三次握手) ---> 数据通信 ---> 断开通信连接(四次挥手)
- 2) 数据传输的方式：以字节流顺序传输；
- 3) 数据传输特点：可靠数据传输(数据无重复到达、无丢失、无差错、无失序)、传输效率低。

2. UDP通信协议

UDP通信协议，也称为用户数据报传输控制协议

无连接的快速数据传输服务

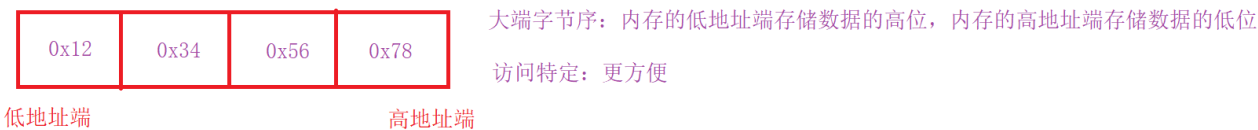
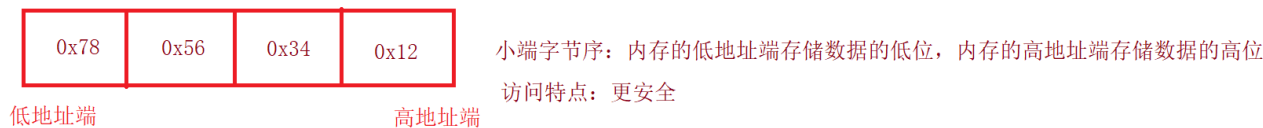
- 1) 通信过程：直接实现数据的通信
- 2) 数据传输的方式：以数据报形式传输
- 3) 数据传输特点：传输效率高、不可靠传输(存在数据帧的丢失，每一个数据帧是完整数据报文)。

2.2 主机字节序和网络字节序

2.2.1 主机字节序

所谓的主机字节序指的是多字节数据在内存中的存储字节序，可以分为小端字节序和大端字节序两种

数据：0x12345678
以字节为单位表示的数据由高位到低位顺序为：0x12、0x34、0x56、0x78



2.2.2 网络字节序

网络字节序产生的原因：是由于在通信的双方主机字节序可以是大端字节序和小端字节序两种方式，那么在通信的时候，作为发送端不能确定接收端的存储方式，同时作为接收端不能确定发送端所发送的字节序，导致无法有效实现任意主机之间数据的通信。为了能够解决问题规定采用统一字节序（大端字节序）进行数据通信，其统一字节序即为网络字节序。

也就是发送端将主机字节序转换为网络字节序发送；接收端将所接收的网络字节序转换为主机字节序。

2.2.3 主机字节序和网络字节序转换函数

```
1 #include <arpa/inet.h>
2 htonl函数实现将无符号32位整型数据的主机字节序转换为网络字节序
3 uint32_t htonl(uint32_t hostlong);
4 htons函数实现将无符号16位整型数据的主机字节序转换为网络字节序
5 uint16_t htons(uint16_t hostshort);
6 ntohl函数实现将无符号32位整型数据的网络字节序转换为主机字节序
7 uint32_t ntohl(uint32_t netlong);
8 ntohs函数实现将无符号16位整型数据的网络字节序转换为主机字节序
9 uint16_t ntohs(uint16_t netshort);
```

2.3 IP地址

在网络通信中，IP地址用于实现点对点通信的主机地址；IPV4和IPV6协议实现IP地址的分配。以下基于IPV4协议标准说明

1. IPV4协议地址用户表示形式：

字符串的点分形式表示：“xxx.xxx.xxx.xxx”，例如：192.168.1.111

2. linux系统中IPV4协议地址的存储

```
1 typedef uint32_t in_addr_t;    /* 重定义数据类型in_addr_t：无符号的32位整型 */
```

```
2 struct in_addr {
3     in_addr_t s_addr;
4 };
5 实质使用的是无符号的32位整型存储
```

3. 地址的转换

```
1 #include <sys/socket.h>
2 #include <netinet/in.h>
3 #include <arpa/inet.h>
4
5 int inet_aton(const char *cp, struct in_addr *inp);
6 功能：将cp指针指向的点分形式的IP地址转换为网络字节序的IP地址存储到inp指向的结构体空间中；
7 in_addr_t inet_addr(const char *cp);
8 功能：将cp指针指向的点分形式IP地址转换为网络字节序的IP地址返回。
9 char *inet_ntoa(struct in_addr in);
10 功能：将网络字节序的IP地址转换为点分形式的IP地址返回。
```

2.4 端口号

端口号标识网络通信的应用程序，TCP通信和UDP通信端口号是独立分配。